

Universitat Autònoma de Barcelona

Master Degree in Modelling for Science and Engineering

Institution / Lab: IIIA-CSIC

Technical Memory Draft Baseline End-to-End Planner Dialogue Flow

Formal architecture and contract section for technical review.

Document type:	Architecture and contract walkthrough
Academic year:	2025-2026
Date:	2026-06-01
Scope:	Runtime flow, JSON contracts, replanning lineage, and dialogue relay behavior.
Repository:	https://github.com/ieverythng/nao-ros4hri-bridge

End-to-End User to Planner Dialogue Flow

1) Purpose

This document is the supervisor-facing and thesis-facing walkthrough for the active planner stack. It is payload-first, but it now separates two world-state views:

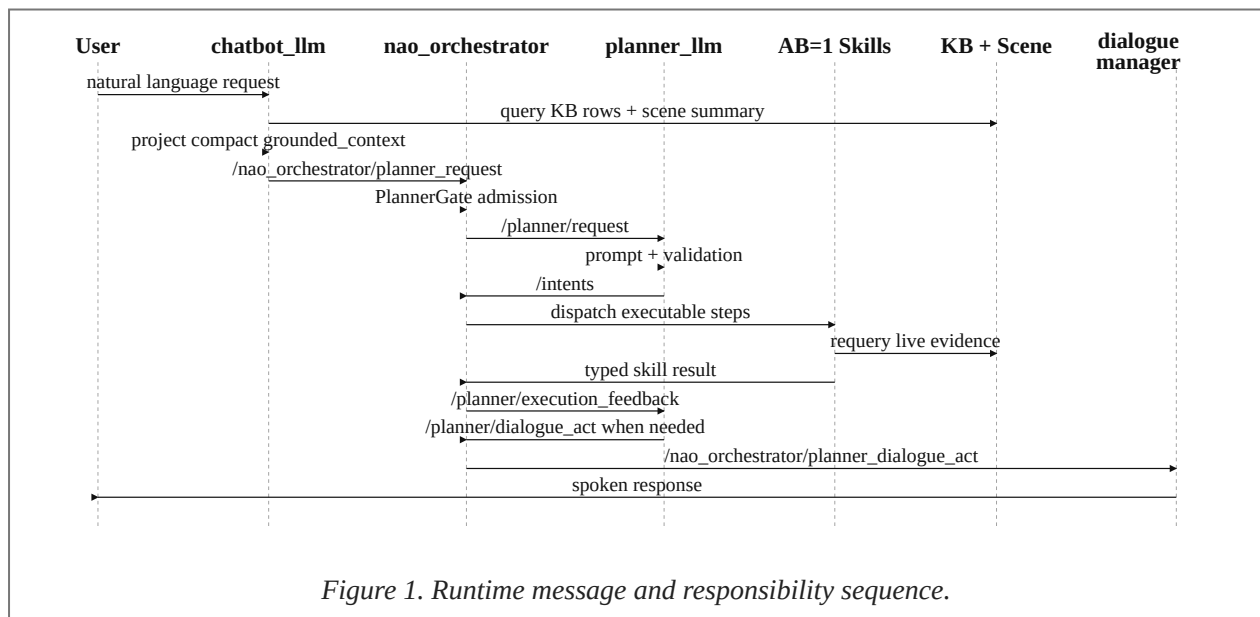
- `grounded_context` : compact scene graph for chatbot/planner LLM calls.
- execution evidence: live skill-local data from `/scene/summary`, KB triples, tracked people, action results, and detector metadata.

The design follows the current neurosymbolic planning pattern used by SayPlan, ProgPrompt, RePLan, and scene-graph replanning work: pass only the task-relevant symbolic subgraph to the LLM, keep symbolic validation outside the LLM, and let execution skills refresh live evidence before claiming results.

Companion references:

- `docs/contracts.md`
- `docs/current_workflow.md`
- `docs/plans/planner_grounding_moe_contract_2026-06-01.md`
- `docs/plans/planner_replan_lineage_adr.md`

2) End-to-End Runtime Flow



Ownership rule:

- `chatbot_llm` owns user-facing interpretation and compact grounding projection.
- `planner_llm` owns planning, validation, retry, replan, and dialogue-act intent.
- `nao_orchestrator` owns deterministic admission, dispatch, and feedback publication.
- AB=1 skills own live world interaction and evidence collection.
- `dialogue_manager` owns speech realization.

3) Contract Map

Contract	Channel	Producer	Consumer
Chatbot turn output	internal turn result JSON	<code>chatbot_llm</code>	<code>chatbot_llm</code> handoff path
Compact grounding	<code>grounded_context</code>	<code>chatbot_llm</code>	<code>chatbot_llm</code> , <code>planner_llm</code>
Planner request envelope	<code>/nao_orchestrator/planner_request</code> , <code>/planner/request</code>	<code>chatbot_llm</code> , admitted by <code>nao_orchestrator</code>	<code>planner_llm</code>
Planner output	<code>/intents</code>	<code>planner_llm</code>	<code>nao_orchestrator</code>
Execution feedback	<code>/planner/execution_feedback</code>	<code>nao_orchestrator</code>	<code>planner_llm</code>
Planner dialogue act	<code>/planner/dialogue_act</code>	<code>planner_llm</code>	<code>nao_orchestrator</code> relay to <code>dialogue_manager</code>
Raw scene/KB evidence	<code>/scene/summary</code> , <code>/kb/query</code> , skill action results	scene/KB/skills	skill-local execution paths

Removed request seams:

- `requested_plan` is not part of the active planner request contract.
- `interaction_mode` is not part of the active planner request contract.
- Legacy fields such as `goal_token`, `world_model_snapshot`, and `world_model_text` remain removed.

4) Chatbot Turn Output

```
{
  "verbal_ack": "Okay, I will do that.",
  "route": "execution",
  "confidence": 0.82,
  "user_intent": {
    "type": "look_at",
    "goal": "look at the person",
    "goal_text": "look at the person",
    "scene_targets": ["person"],
    "request_kind": "new_goal"
  }
}
```

Variables:

- `verbal_ack`: optional immediate acknowledgement text.
- `route`: `dialogue` | `knowledge_query` | `execution`.
- `confidence`: route/intent confidence in `[0.0, 1.0]`.
- `user_intent.type`: normalized intent label.
- `user_intent.goal` / `goal_text`: planner-facing objective text.
- `user_intent.scene_targets`: compact target labels or entity IDs.
- `user_intent.request_kind`: request transition class.

5) Compact Grounded Context

The LLM-facing world state is a concise scene graph, not a raw RDF dump.

```
{
  "grounded_context": {
    "entities": [
      {
        "id": "cup_jrjic",
        "label": "cup",
        "kind": "object",
        "class": "Cup",
        "visible": true,
        "relations": [
          {"predicate": "rdf:type", "object": "Cup"},
          {"predicate": "dbp:color", "object": "blue"},
          {"predicate": "oro:isOn", "object": "table_1"}
        ]
      },
      {
        "id": "anonymous_person_ehfbf",
        "label": null,
        "kind": "person",

```

```

    "class": "Human",
    "visible": true,
    "relations": [
      {"predicate": "rdf:type", "object": "Human"}
    ]
  },
  ],
  "counts": {"entities": 2, "objects": 1, "people": 1}
}
}

```

Policy:

- `id` is the stable entity handle.
- `label` is the human-readable class/name hint and may be `null` for anonymous people.
- `kind` is `object` or `person`.
- `class` is the compact semantic class.
- `visible` is the current visual grounding flag.
- `relations` only carries prioritized semantic predicates for LLM reasoning.

Prioritized predicates:

- Always include one `rdf:type` relation when available.
- Include user-meaningful simulator/KB predicates when present: `dbp:name`, `dbp:color`, `oro:isAt`, `oro:isOn`, `oro:contains`, `foaf:knows`.
- Drop noisy/default prompt relations such as provenance, detector score, coordinates, backend, source, and `last_seen_*`.
- Keep raw triples available to skill-local execution paths when a skill needs exact RDF.

6) Grounding Projection Pipeline

```

flowchart LR
    A["/scene/summary"] --> P["project_llm_grounded_context"]
    B["tracked people"] --> P
    C["/kb/query rows and RDF triples"] --> P
    D["state_t0 flag (disabled by default)"] --> P
    P --> G["compact grounded_context"]
    A --> X["execution_context / skill-local raw evidence"]
    C --> X
    X --> S["AB=1 skill requery and validation"]

```

Rules:

1. Backend seams stay intact: `/scene/summary`, `/kb/query`, `/kb/revise`, tracked people, and skill action results are not rewritten by the projection.
2. The compact projection is deterministic and JSON-native.
3. Text-derived hydration is fallback-only when no structured scene/KB entities exist.
4. `state_t0` remains behind `grounded_context_include_state_t0`; default is disabled.
5. Planner-only details such as coordinates and recency are not part of the default chatbot/planner LLM view.

7) Planner Request Envelope

Transport-level envelope shape as serialized in the `Intent.data` field:

```
{
  "request_id": "turn_123",
  "goal_id": "goal_turn_123",
  "parent_goal_id": "",
  "supersedes_goal_id": "",
  "request_kind": "new_goal",
  "goal_text": "look at the person and report completion",
  "normalized_intents": ["look_at"],
  "scene_targets": ["person"],
  "dialogue_context": ["assistant:Okay, I will do that."],
  "grounded_context": {
    "entities": [
      {
        "id": "anonymous_person_ehfbf",
        "label": null,
        "kind": "person",
        "class": "Human",
        "visible": true,
        "relations": [{"predicate": "rdf:type", "object": "Human"}]
      }
    ],
    "counts": {"entities": 1, "objects": 0, "people": 1}
  },
  "planner_mode": "default",
  "dialogue_turn_id": "role:turn"
}
```

Variables:

- Identification and lineage: `request_id`, `goal_id`, `parent_goal_id`, `supersedes_goal_id`.
- Transition control: `request_kind`.
- Planning objective: `goal_text`, `normalized_intents`, `scene_targets`.
- Context: `dialogue_context`, `grounded_context`.
- Runtime mode: `planner_mode`, `dialogue_turn_id`.

The planner prompt receives this compact payload plus execution feedback, skill registry, allowed step types, allowed skill names, and output contract.

8) Planner LLM Internals

```

flowchart TD
    R["PlannerRequest.from_payload"] --> G["PlannerGate admission  
already completed"]
    G --> E["PlannerEngine.plan_request"]
    E --> RB["Rule-based simple motion?"]
    RB -- yes --> BP["Build deterministic rule plan"]
    RB -- no --> PR["Build LLM prompt payload"]
    PR --> L["LLM provider"]
    L --> V["Parse JSON + validate steps"]
    V -->|valid| OUT["build_plan_payload"]
    V -->|invalid| RETRY["validation retry prompt"]
    RETRY --> V
    V -->|still invalid| FAIL["fail/clarify payload"]
    BP --> OUT
    OUT --> I["/intents"]

```

Planner prompt payload contains:

- `request`: compact planner request.
- `goal_id`, `plan_version`.
- optional `state_t0` only when explicitly enabled by flag.
- `execution_feedback` during replans.
- `skill_registry`, `allowed_step_types`, `allowed_skill_names`, `allowed_motion_objects`.
- `output_contract`.

Planner prompt payload does not contain:

- raw `user_text` in normal operation.
 - raw detector coordinates or scores.
 - full RDF dumps.
 - removed request seams.
-

9) Planner Output

```

{
  "plan": {
    "goal_id": "goal_turn_123",
    "plan_id": "plan_123",
    "plan_version": 2,
    "status": "planning",
    "validation_status": "draft",
    "user_facing_reason": "",
    "replan_hint": "",
    "retry_budget": 1,
    "scene_targets": ["person"],
    "context_ref": {
      "captured_at_sec": 0.0,
      "observer": "",
      "backend": ""
    },
  },
  "communication_policy": {
    "emit_acknowledge": false,
    "emit_progress": false,
    "emit_completion": true,
    "emit_failure": true
  },
  "communication_policy_source": "planner_engine:plan",
  "steps": [
    {
      "id": "step_1",
      "type": "skill",
      "name": "scan",
      "args": {},
      "requires": [],
      "on_failure": "replan",
      "retry_budget": 0
    },
    {
      "id": "step_2",
      "type": "skill",
      "name": "report_result",
      "args": {},
      "requires": ["step_1"],
      "on_failure": "fail",
      "retry_budget": 0
    }
  ]
}

```

Variables:

- Plan lineage: `goal_id`, `plan_id`, `plan_version`.
- Plan state: `status`, `validation_status`, `user_facing_reason`, `replan_hint`.
- Failure detail: `failure_reason` is present only when there is an actual failure, invalid output, or terminal blocked condition.
- Resilience: `retry_budget`.

- Traceability: `scene_targets`, `context_ref`.
- Communication: `communication_policy`, `communication_policy_source`.
- Execution content: `steps[]` with `id`, `type`, `name`, `args`, `requires`, `on_failure`, `retry_budget`.

10) Communication Policy

`communication_policy` is planner dialogue-act permission, not direct speech execution.

- `emit_acknowledge`: allow a planner-owned acknowledgement dialogue act on plan acceptance.
- `emit_progress`: allow planner-owned progress dialogue acts on step start/progress.
- `emit_completion`: allow planner-owned completion dialogue acts on plan completion.
- `emit_failure`: allow planner-owned failure/help dialogue acts on execution failures.
- `communication_policy_source`: deterministic provenance showing where the final policy was resolved, usually `planner_engine:<mode>`.

Defaults come from `planner_common.normalize_communication_policy`. The planner may override policy flags in its JSON output; the shared contract normalizes them before supervisor use.

11) Supervisor Lifecycle

```
stateDiagram-v2
    [*] --> idle
    idle --> planning: accepted request
    planning --> executing: valid plan
    planning --> waiting_user: clarify
    planning --> failed: invalid/fail
    executing --> replanning: step failed + retry budget
    executing --> waiting_user: needs user input
    executing --> completed: all steps succeeded
    executing --> failed: terminal failure
    replanning --> executing: new valid plan
    replanning --> waiting_user: no safe continuation
    waiting_user --> planning: clarification_answer
    completed --> idle
    failed --> idle
```

Supervisor responsibilities:

1. Track `goal_id`, `plan_id`, and `plan_version`.

2. Admit new plans and suppress duplicate plan versions.
 3. Dispatch only executable steps through the orchestrator.
 4. Consume execution feedback and decrement retry budget.
 5. Ask for replans when failure policy and budget allow.
 6. Emit planner dialogue acts for clarification, progress, completion, or failure according to `communication_policy`.
-

12) Scan and Report Evidence Flow

flowchart TD

```

    A["User: what do you see?"] --> B["chatbot_llm compact grounded_context"]
    B --> C["planner_llm plans scan + report_result"]
    C --> D["nao_orchestrator dispatches scan"]
    D --> E["scan skill requeries scene/KB"]
    E --> F["scan result payload with summary_text"]
    F --> G["orchestrator stores latest result"]
    G --> H["report_result step with empty args"]
    H --> I["orchestrator reuses latest live result"]
    I --> J["execution_feedback to planner"]
  
```

Critical guardrail:

- The planner must not prefill `report_result.args.summary_text` after `scan`.
- The scan step owns fresh perception.
- `report_result` with empty args tells the orchestrator to report the latest live skill result.
- `summary_text` is only allowed for known non-perception facts or already completed non-perception results.

This prevents a stale prompt snapshot from being mistaken for a fresh scan.

13) Execution Feedback

```

{
  "goal_id": "goal_turn_123",
  "plan_id": "plan_123",
  "plan_version": 2,
  "intent": "raw_user_input",
  "source": "planner_llm",
  "event_type": "step_succeeded",
  "status": "succeeded",
  "reason": "",

```

```

"validation_status": "valid",
"replan_hint": "",
"retry_budget": 1,
"blocking": false,
"unmet_preconditions": [],
"needs_user_input": false,
"scene_targets": ["person"],
"validation_errors": [],
"timestamp_sec": 1777040000.0,
"result_summary": "Look-at completed for person_1",
"result_payload": {
  "skill": "look_at",
  "target": "person_1",
  "target_kind": "people",
  "target_found": true,
  "summary_text": "Look-at completed for person_1",
  "confidence_policy": "grounded_current_observation"
},
"step": {
  "id": "step_1",
  "type": "skill",
  "name": "look_at",
  "retry_budget": 0,
  "on_failure": "replan",
  "requires": []
}
}

```

Evidence fields:

- `result_summary`: compact human-readable evidence summary.
- `result_payload`: structured skill result.
- `scene_targets`: targets used for replan context.
- `blocking`, `needs_user_input`, `unmet_preconditions`: supervisor decision hints.

14) Planner Dialogue Act

```

{
  "goal_id": "goal_turn_123",
  "plan_id": "plan_123",
  "plan_version": 2,
  "act": "ask_clarification",
  "priority": "normal",
  "await_user_response": true,
  "reason": "missing target object",
  "text_hint": "Which person should I look at?",
  "slots_needed": ["target"],
  "context": {
    "scene_targets": ["person"],
    "status": "waiting_user"
  }
}

```

```
}
}
```

Allowed `act` values:

- `acknowledge`
- `progress_update`
- `ask_clarification`
- `ask_for_help`
- `explain_failure`
- `notify_completion`
- `notify_cancellation`

Dialogue acts travel through `nao_orchestrator` to `dialogue_manager`. The planner does not speak directly.

15) Chatbot / Planner / Skill Ownership Map

flowchart LR

```

    C["chatbot_llm\nuser-facing grounding + planner handoff"] -->
    P["planner_llm\nplan + supervise"]
    P --> O1["nao_orchestrator\nadmit + dispatch + feedback"]
    O1 --> S1["scan\nfresh perception"]
    O1 --> S2["look_at\ntarget resolution"]
    O1 --> S3["navigate_to\nmovement evidence"]
    O1 --> S4["report_result\nlatest result speech payload"]
    S1 --> K1["scene/KB/tracked people"]
    S2 --> K2[" "]
    S3 --> K3[" "]
    S4 --> O2[" "]
    O2 --> D["dialogue_manager\nspeech realization"]
  
```

Methodology:

1. Keep LLM context compact and semantically rich.
2. Keep raw sensor/KB evidence out of every prompt by default.
3. Validate planner output outside the LLM.
4. Require skills to requery live state before reporting execution success.
5. Use planner dialogue acts for coordination, not direct speech generation.

16) Replanning and Join Semantics

flowchart TD

```

A["Receive /planner/request"] --> B{"Existing active goal?"}
B -- No --> C["Create new goal lineage"]
B -- Yes --> D{"request_kind"}
D -- new_goal --> E["Supersede or reject based on lineage"]
D -- goal_update --> F["Keep goal_id continuity"]
D -- clarification_answer --> F
D -- cancel_request --> G["Cancel active goal"]
C --> H["Publish /intents"]
E --> H
F --> H
H --> I["Execute steps"]
I --> J["Publish execution feedback"]
J --> K{"Need replan?"}
K -- Yes --> L["Increment plan_version and replan"]
K -- No --> M["Continue or complete"]
L --> H
M --> H

```

Current policy summary:

1. Single active-goal queue model.
 2. `goal_id` keeps goal continuity.
 3. `plan_id` + `plan_version` encode plan lineage.
 4. Mid-join when active `step_id` is present in the new plan.
 5. Front-join from first pending step when mid-join cannot be resolved.
 6. Duplicate suppression compares `goal_id`, `plan_id`, and `plan_version`.
-

17) Live Supervisor Demo Script

1. Trigger one command and show chatbot output (`route=execution`).
2. Show compact `grounded_context` on the planner request.
3. Show admitted request on `/planner/request`.
4. Show planner output on `/intents` and highlight lineage fields.
5. Run one scan/report case and show that `report_result.args` is empty.
6. Run one failure/clarification path.
7. Show execution feedback payload and resulting dialogue act payload.
8. Close with ownership map: chatbot grounds, planner plans, orchestrator dispatches, skills provide live evidence, dialogue manager speaks.